

Hazelcast (Executor 模块) 源码阅读报告

2022k8009925009 王敬华

Part 0.什么是 Hazelcast?

Hazelcast 是由 Hazelcast 公司开发的一款开源的分布式内存级别的缓存数据库，可以为基于 JVM 环境运行的各种应用提供分布式集群和分布式缓存服务。

Hazelcast 提供了一个分布式内存计算框架，可以通过内存来存储和计算数据，从而加速应用程序的响应和处理速度。它最常见的功能包括分布式缓存、数据存储、任务执行等，用于提高系统的性能和可扩展性。（好官方，没看懂没关系，本人之前没学过 java，也没上过任何关于数据库或者分布式的课，别问我为什么选这个，我也不知道可能是顺眼吧）

（说人话）Hazelcast 是一个可以帮助你多台计算机上同时处理数据和计算任务的工具，通俗地讲，Hazelcast 就像是一个让许多计算机一起工作的“指挥中心”，它帮助它们共同处理数据和完成任务，让整个系统的速度更快、效率更高。Hazelcast 它最常见的功能包括分布式缓存、数据存储、任务执行等。

Hazelcast 提供了对很多 Java 接口的分布式实现，如 Map,ExecutorService, Queue, Lock 以及 JCache。它以一个 JAR 包的形式提供服务，并且提供了 Java, C/C++, .NET 以及 REST 客户端。

在应用时，可以将 Hazelcast 的 jar 包直接嵌入到任何使用 Java、C++、.NET 开发的产品中，我们只需要在应用中引入一个 jar 包，进行简单的配置和编码就可以实现。

下面简单介绍 Hazelcast 的主要特点：

- 1.分布式缓存：Hazelcast 提供了基于内存的分布式缓存功能，可以在集群节点间共享数据，提高访问速度。
- 2.分布式数据结构：提供了多种分布式集合，如分布式 Map、Queue、List 等。
- 3.分布式任务执行：支持分布式的任务执行功能，可以在多个节点上并行处理任务。
- 4.数据分区和容错：支持将数据分区存储在不同节点上，并进行复制，确保数据在节点故障时不会丢失。

下面简单介绍在 Java 中如何使用 Hazelcast

在 Java 中使用 Hazelcast 的具体步骤如下：

1. 引入依赖：

在 Java 项目中，需要添加 Hazelcast 的依赖项。可以使用 Maven 或 Gradle 等构建工具来管理依赖关系。以下是 Maven 的示例配置：

```
<dependency>
  <groupId>com.hazelcast</groupId>
  <artifactId>hazelcast</artifactId>
  <version>4.2</version></dependency>
```

2. 创建 Hazelcast 实例：

在代码中，首先需要创建 Hazelcast 实例来启动一个 Hazelcast 集群。以下是一个创建 Hazelcast 实例的示例代码：

```
import com.hazelcast.core.Hazelcast;import com.hazelcast.core.HazelcastInstance;

public class HazelcastExample {
```

```

public static void main(String[] args) {
    // 创建Hazelcast 实例      HazelcastInstance hazelcastInstance =
Hazelcast.newHazelcastInstance();

    // 执行其他操作...    }}

```

3. 使用分布式数据结构:

Hazelcast 提供了各种分布式数据结构, 如 Map、Queue、List 等。可以使用这些数据结构来存储和访问分布式数据。以下是一个使用 Hazelcast Map 的示例代码:

```

import com.hazelcast.core.Hazelcast;import
com.hazelcast.core.HazelcastInstance;import com.hazelcast.core.IMap;

public class HazelcastExample {
    public static void main(String[] args) {
        // 创建Hazelcast 实例      HazelcastInstance hazelcastInstance =
        Hazelcast.newHazelcastInstance();

        // 获取分布式Map      IMap<String, String> map =
        hazelcastInstance.getMap("myMap");

        // 存储数据      map.put("key1", "value1");
        map.put("key2", "value2");

        // 获取数据      String value1 = map.get("key1");
        String value2 = map.get("key2");

        System.out.println("Value1: " + value1);
        System.out.println("Value2: " + value2);
    }
}

```

4. 运行应用程序:

可以通过运行上述代码来启动你的应用程序, 并使用 Hazelcast 进行分布式数据存储和访问。根据具体需求可以使用其他 Hazelcast 的功能和 API 来完成更多的任务。

我选择分析的模块是 Hazelcast 的 Executor 模块, 这是一个分布式任务执行器, 允许你在 Hazelcast 集群中的不同节点上并行执行任务。它类似于 Java 的 ExecutorService (这是 Java 的并发编程 API 的一部分, 有点像 c 语言的 pthread, 简化了线程管理、任务调度和执行的过程), 但 Hazelcast 的 Executor 是分布式的, 支持将任务发送到集群的一个或多个节点上执行。这个模块特别适用于需要利用多节点的计算能力来处理大量任务的场景, 比如大规模数据处理、实时分析等。

实际上, 通过后面查阅官网相关信息, 得知层次图中明确了其主要接口 IexecutorService 就是继承自 java.util.concurrent.ExecutorService 中。然而, 显然 Hazelcast Executor 模块不是单纯的一个调库, hazelcast.executorservice 的独到之处是: 它实现了分布式环境中的异步任务。

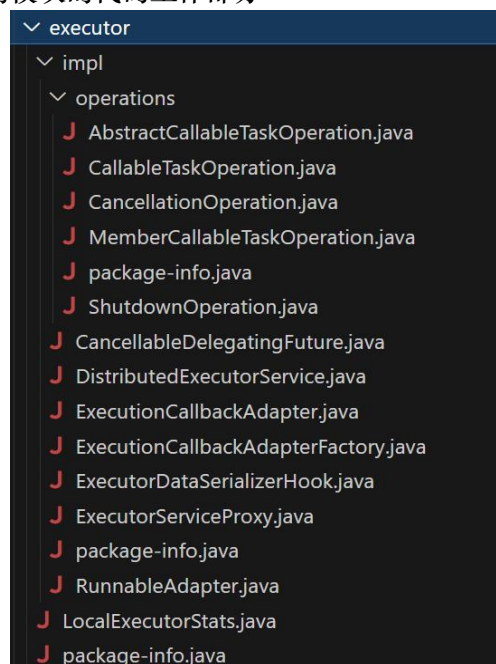
- com.hazelcast.core.EntryListener<K,V> (also extends com.hazelcast.map.listener.EntryListener<K,V>, com.hazelcast.map.listener.EntryExpiredListener<K,V>, com.hazelcast.map.listener.MapEvictedListener)
- com.hazelcast.map.listener.MapEvictedListener
 - com.hazelcast.core.EntryListener<K,V> (also extends com.hazelcast.map.listener.EntryListener<K,V>, com.hazelcast.map.listener.EntryExpiredListener<K,V>, com.hazelcast.map.listener.MapEvictedListener)
- com.hazelcast.spi.eviction.EvictableEntryView<K,V>
 - com.hazelcast.core.EntryView<K,V>
- com.hazelcast.core.ExecutionCallback<V>
- java.util.concurrent.Executor
 - java.util.concurrent.ExecutorService
 - com.hazelcast.core.IExecutorService (also extends com.hazelcast.core.DistributedObject)
- com.hazelcast.core.HazelcastInstance
- com.hazelcast.core.HazelcastInstanceAware
- com.hazelcast.core.ICacheManager
- com.hazelcast.core.IndeterminateOperationState
- com.hazelcast.core.LifecycleService
- com.hazelcast.core.ManagedContext
- com.hazelcast.core.MultiExecutionCallback
- com.hazelcast.core.Offloadable
- com.hazelcast.core.ReadOnly
- java.io.Serializable
 - com.hazelcast.core.IFunction<T,R>
- com.hazelcast.core.TypeConverter

在提异步任务之前，我们首先来探究一下 executor 框架：Executor 框架可以让我们轻松的异步执行任务，比如数据库查询，复杂计算和图像渲染等。ThreadPoolExecutor 是 java Executor 框架的默认实现，它主要为在单 JVM 内运行而设计。在分布式系统中，ThreadPoolExecutor 可能不是期望的 Executor 实现，因为提交任务和执行任务的 JVM 是不同的。

为了解决这个问题，Hazelcast 提供了 IExecutorService 用于在分布式环境中执行异步任务，其中依然实现了 java.util.concurrent.ExecutorService 接口，为需要计算和数据处理能力的应用程序提供服务。为了达到异步分布式执行的目的，IExecutorService 中的任务必须支持序列化以在不同的 JVM 之间通过网络传输

注：Java 的 Executor 框架是一个用于并发编程的工具集，提供了一个更高级别的 API 来管理和控制线程，而不是直接使用底层的 Thread 类。这个框架的核心就在于它简化了多线程任务的提交、调度和执行流程。

下面这张图是我分析的模块的代码主体部分：



Part 1.功能分析和建模

1. Hazelcast Executor 模块概述

Hazelcast 的 Executor 模块提供了一个分布式任务执行器，支持在 Hazelcast 集群中并行执行任务。它允许开发者将任务提交到集群中的某一节点或一组节点上执行，从而实现分布式计算、任务调度、并发控制等。该模块在大规模分布式系统和实时数据处理中尤为重要，能够有效利用集群节点的资源。

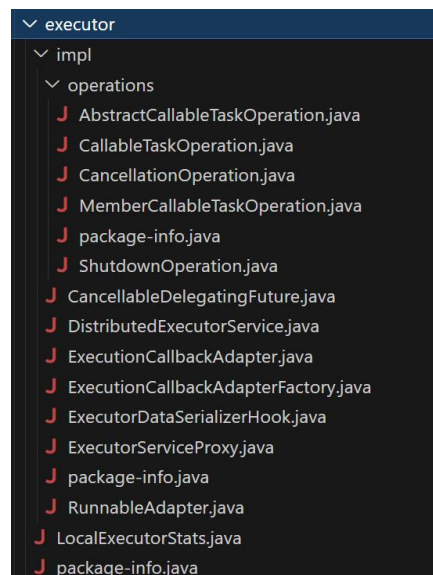
其主要功能包括：

分布式任务执行：将任务分发到多个节点上并行执行，提高计算性能。

任务调度与管理：可以指定任务在特定节点或所有节点执行，灵活控制任务分配。

容错处理：如果某个节点出现故障，Executor 模块可以重新分配任务，保证任务的最终完成。

Hazelcast Executor 模块代码框架如图所示：



impl 文件夹：impl 文件夹包含功能实现的操作逻辑， executor 整体实现在这里的代码完成。

Operations 文件夹：在 Hazelcast 中，operations 是执行特定任务或命令的内部方法，

对于 executor 模块来说，这些 operation 负责：

描述任务的执行：例如，如何执行一个 Runnable 或 Callable 任务。管理任务的生命周期：任务什么时候开始、什么时候结束，以及如何报告结果或错误。处理任务数据传输：因为任务是在分布式环境中执行，它们需要被序列化（转换为一种 可以发送的格式）并发送到目标节点，然后在目标节点上反序列化和执行。

那么 impl 中的文件如何与 Operations 关联？

impl 文件夹中的类会具体实现某些 operation。一些实现还包含如何在节点上实际运行业务逻辑的代码。impl 是该模块的核心实现部分。它定义了如何在分布式集群环境下执行、传输和管理任务。而 operation 是这些逻辑的具体执行者。

2. Hazelcast Executor 模块需求建模

围绕着 Hazelcast 的 Executor 模块的主要功能，我们对其进行功能建模分析

[用例名称]

分布式任务执行

[用例描述]

通过 Hazelcast 的 Executor 模块将计算任务分发到 Hazelcast 集群的不同节点上并行执行，完成高效的分布式计算，并能够在任务执行后收集结果。

[场景]

Who: 分布式系统的开发者、数据处理工程师、后台服务。

Where: 在 Hazelcast 集群环境中运行，包括分布式缓存、数据处理、微服务集群等场景。

When: 需要执行并行计算任务时，如批量数据处理、数据分析、实时数据监控等。

[用例价值]

Hazelcast 的 Executor 模块提供了分布式任务执行的能力，使开发者可以在集群中的多个节点上并行处理计算任务，最大化资源利用率，降低单点压力，提高系统的整体处理效率。同时，模块还支持自动处理节点故障，使得任务执行更具容错性，增强系统的可靠性。

[约束和限制]

任务可序列化：所有提交的任务必须实现 Serializable 接口，以便任务可以在网络上传输到不同的节点。

任务执行环境依赖：任务所需的环境（如特定的库、资源）必须在所有节点上都可用，否则可能导致执行失败。

节点负载均衡：需要合理的负载均衡策略，以确保任务在节点间的合理分配，避免部分节点过载。

故障重试机制：当节点故障或任务失败时，需要设计合理重试和重新分配机制，以确保任务最终完成。

同步与异步结果处理：使用 Future 对象时，应处理好异步任务的结果，避免因阻塞导致性能下降。

[示例用例名称]

特定节点任务分配

[用例描述]

通过 Hazelcast 的 Executor 模块，将一个计算任务分配到 Hazelcast 集群中的指定节点上执行。

[场景]

Who: 系统管理员、任务分发管理者。

Where: 在 Hazelcast 集群中指定的节点上执行，例如对特定数据源的数据处理或节点健康检查。

When: 需要将任务仅分配给特定节点时。

[用例价值]

指定任务执行节点可以实现任务的精准调度，适用于需要特定节点资源或环境的任务，提高集群的可控性和任务分配的灵活性。

[约束和限制]

节点选择：需要确保目标节点在线，且具备执行任务的资源，防止任务失败。

节点状态检测：在任务执行前，需要确认节点的健康状态，以保证任务能够成功执行。

3. Hazelcast Executor 模块的工作流程

Hazelcast Executor 模块的工作流程可以描述为以下几个步骤：

(1) 初始化：

DistributedExecutorService 初始化时注册组件与需要的服务。加载并缓存执行器的配置。

(2) 任务提交：

用户通过 ExecutorServiceProxy 提交任务。根据需求，可能是直接执行或以 RunnableAdapter 形式。

任务根据执行策略由集群中的节点处理，确定执行节点后对应的 CallableTaskOperation 被序列化并发送到目标节点。

(3) 任务执行：

在目标节点，Processor 将任务包装进 FutureTask 并执行。

根据配置监控与统计任务的执行状态，记录执行时间与结果。

(4) 结果处理：

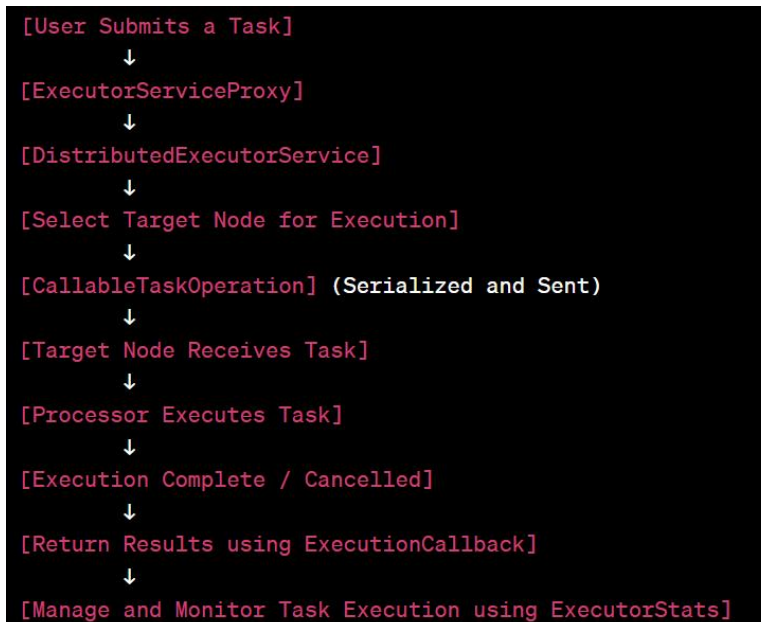
ExecutionCallbackAdapterFactory 管理任务的回调。每个任务完成后，调用注册的回调函数处理结果。

如果任务被取消，CancellableDelegatingFuture 将处理取消逻辑。

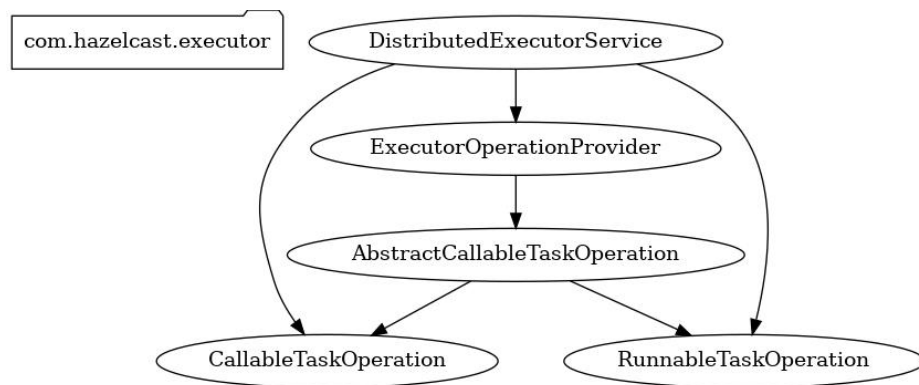
(5) 销毁与重置：

通过 DistributedExecutorService 的 API，可以停止执行器或者关闭特定任务/服务。资源释放时清理相关缓存与配置管理。

工作流程示意图如下：

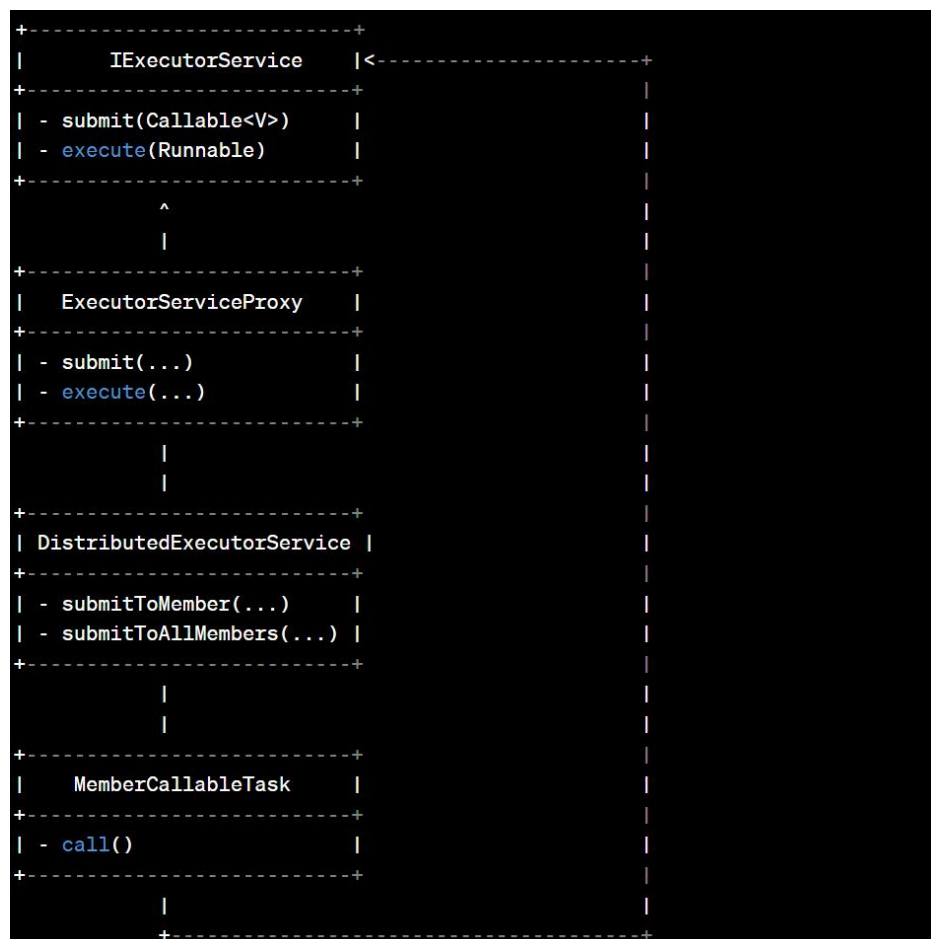


用图像可以表示为：



4. Hazelcast Executor 模块的建模

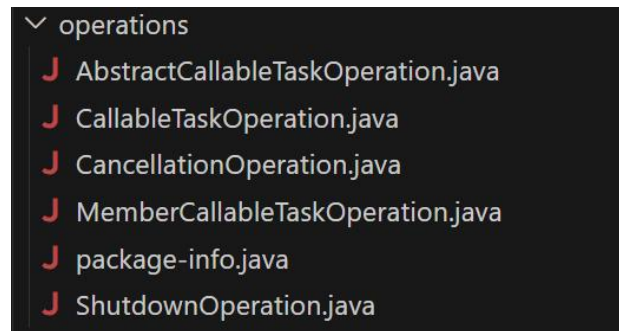
在建模过程中，可以将 `IExecutorService` 视作集群范围内的执行管理器，`DistributedExecutorService` 则是具体的执行调度器，而 `MemberCallableTask` 是执行的具体逻辑。以下是一个简化的类图的描述，用于理解模块的整体结构和调用关系：



Part 2.核心流程设计分析

1.内部方法分析

此部分主要涉及到.hazelcast.impl.operations 文件夹中的 5 种方法(5 个 java 文件): Hazelcast 中的每个任务都被封装成一个操作 (Operation)。操作通常是一个指令,告诉 Hazelcast 系统要做什么事情。定义了一个抽象操作类,它代表了 Hazelcast 中可调用任务的一部分。



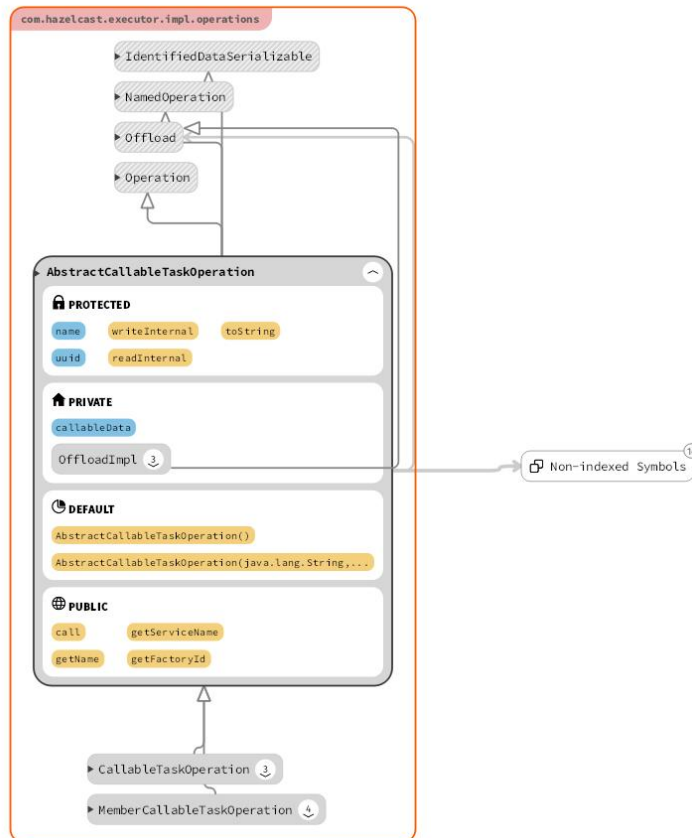
(1)内部方法分析-AbstractCallableTaskOperation()

继承了 Operation 方法,实现了命名操作和识别数据序列化,同时内部使用了 UUID 作为进程保护

name: 表示任务的名称。

uuid: 表示任务的唯一标识符,UUID 是一种全球唯一的标识符。

callableData: 存储与任务相关的数据,这些数据将会传递给具体的任务执行者。



主要功能：

利用 ObjectDataInput 和 ObjectDataOutput 将数据序列化交互,同时实现了 offload 功能,当负载过重的时候,可以用这个方法实现负载转移。

序列化实现：

Java 提供了一种对象序列化的机制,该机制中,一个对象可以被表示为一个字节序列,该字节序列包括该对象的数据、有关对象的类型的信息和存储在对象中数据的类型。

将序列化对象写入文件之后,可以从文件中读取出来,并且对它进行反序列化,也就是说,对象的类型信息、对象的数据,还有对象中的数据类型可以用来在内存中新建对象。

通过 import 下列两个包实现了序列化：

```
import com.hazelcast.internal.nio.IOUtil;
import com.hazelcast.internal.util.UUIDSerializationUtil
```

(2)内部方法分析-CallableTaskOperation

CallableTaskOperation 继承了 AbstractCallableTaskOperation 抽象类的实现,参数基本相似。

```
public final class CallableTaskOperation extends AbstractCallableTaskOperation
    implements IdentifiedDataSerializable, MutatingOperation {
```

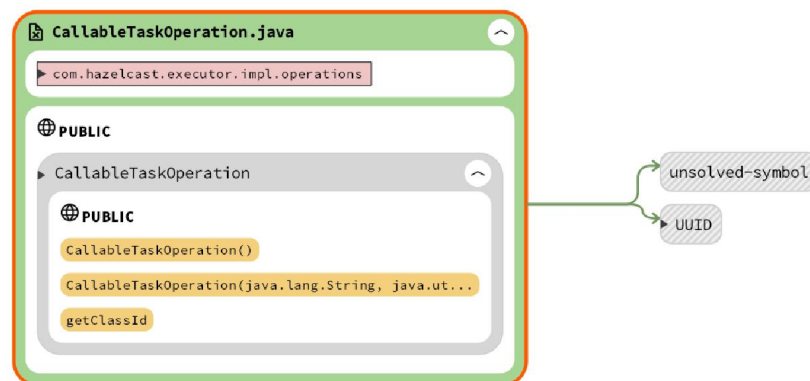
CallableTaskOperation 类是一个表示“可调用任务操作”的具体实现,它是 Hazelcast 中分布式执行服务的一部分。通过继承 AbstractCallableTaskOperation,它继承了父类的大部分功能(比如任务的名称、UUID、数据序列化等),并且实现了必要的

接口来支持序列化和状态变更操作。

IdentifiedDataSerializable 接口让这个类能够在网络中传输,保证数据的安全和正确性。同时接口支持对象的序列化。

MutatingOperation 接口表明这个操作会改变系统的状态,可能会对集群中的某些数据进行修改。

getClassId 方法是 IdentifiedDataSerializable 接口的一部分,用来返回这个类的唯一 ID。这个 ID 用于在序列化时标识对象类型,确保不同类型的对象能够正确地被反序列化。



(3)内部方法分析-CallableTaskOperation

MemberCallableTaskOperation 用于在分布式系统中的一个特定机器节点上执行可调用任务（任务即 Callable），这种任务在特定的集群成员（节点）上执行。如果节点崩溃或离开，系统会通过异常处理机制对任务失败进行处理。

MemberCallableTaskOperation 同样继承了 AbstractCallableTaskOperation 抽象类的实现，但特殊的是，这里额外多了一个报错，当 member 不存在或者对象不是 member 时就会报错。

onInvocationException 方法：

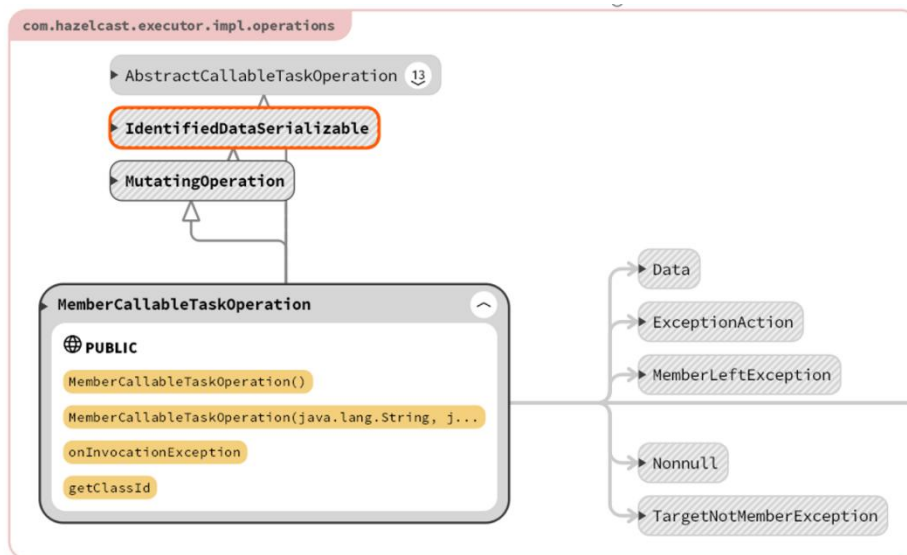
在任务执行期间，如果遇到异常（错误、问题），这个方法会被调用。

特定异常处理：

MemberLeftException：如果有成员（机器）离开任务执行环境。

TargetNotMemberException：如果目标成员不在集群中（可能已经被移除）。

当检测到这些特定异常时，操作将抛出新的异常，以便调用者能处理这些问题。



(4)内部方法分析-CancellationOperation

CancellationOperation 类的设计是为了在 Hazelcast 集群中提交任务取消请求。换句话说，当你希望停止一个已经提交到分布式集群中执行的任务时，会用这个类来发起取消请求。

uuid: 任务的唯一识别码，用于确定需要撤销的任务是哪一个。

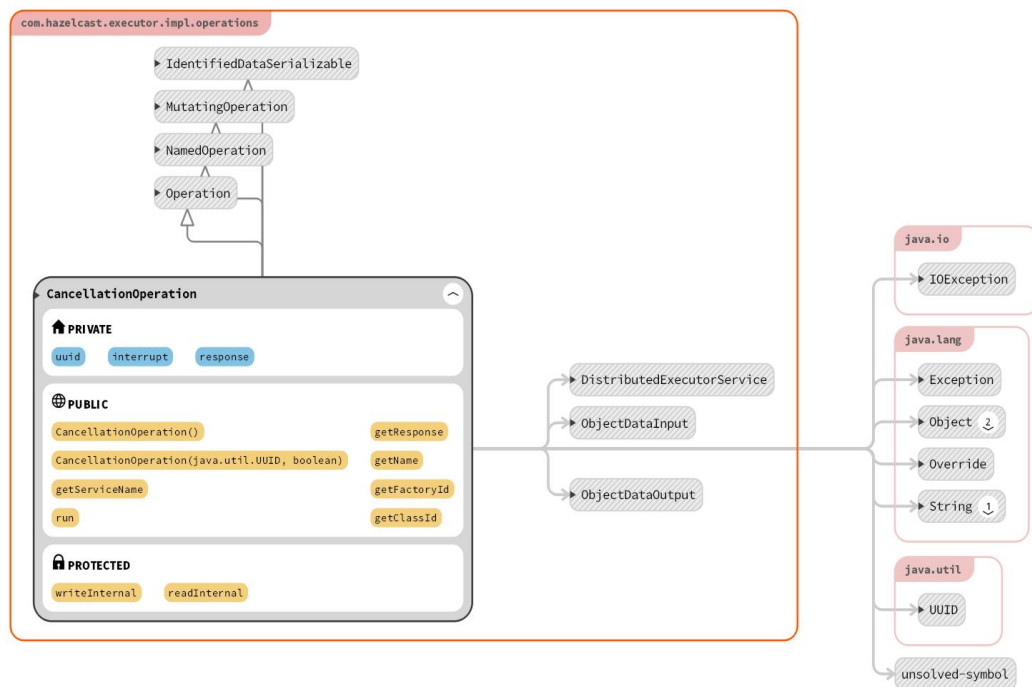
boolean interrupt: 指示是否可以中断正在运行的任务。

boolean response: 用于保存取消操作的结果（成功或失败）。

run 方法: 执行取消任务的操作。它从 DistributedExecutorService 中调用 cancel() 方法来执行实际的任务取消。

getResponse: 返回操作结果。成功取消返回 true，否则返回 false。

writeInternal 和 readInternal 方法: 负责将对象转换为字节流并从字节流中生成对象。这对于 Hazelcast 的分布式环境中传输消息至关重要。



(5)内部方法分析-ShutdownOperation

没有定义自己的变量，全部调用的其他类的方法
执行主要靠下段代码：

```
public void run() throws Exception {
    DistributedExecutorService service = getService();
    service.shutdownExecutor(getName());
}
```

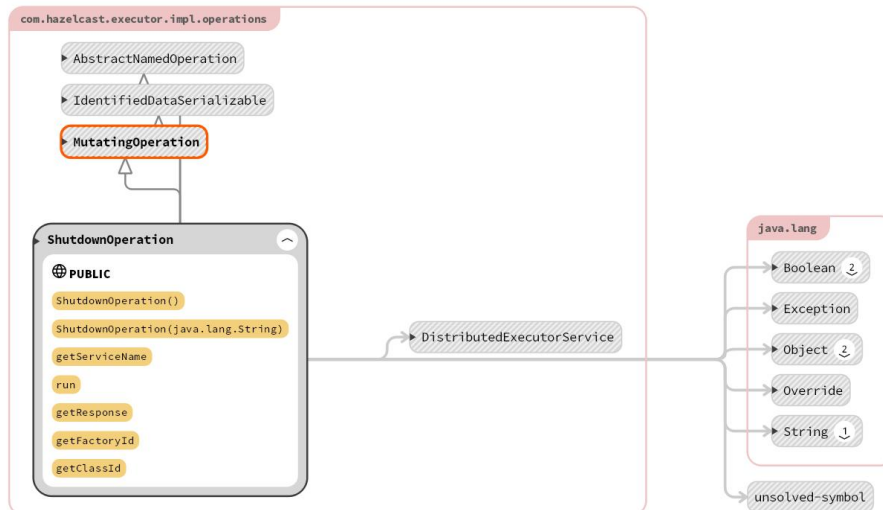
`ShutdownOperation` 类的设计目的是为了在 Hazelcast 集群中关闭一个名为“执行器”的组件。执行器就像是分布式系统中的一个任务管理器，可以安排和执行任务。因此，关闭操作就类似于停止这个任务管理器的运行,调用 `Dexecutor` 中的关闭方法完成关闭

`run` 方法：执行关闭操作的核心逻辑。

首先获取 `DistributedExecutorService` 服务的实例，然后调用 `service.shutdownExecutor(getName());` 方法，根据执行器的名字来关闭相应的执行器。

`getServiceName` 方法：返回相应服务的名字。在这里，返回的名字是 `DistributedExecutorService.SERVICE_NAME`，标识这个操作与 Hazelcast 的分布式执行器服务相关联。

`getFactoryId` 和 `getClassId` 方法：分别返回工厂 ID 和类 ID，表明这个操作可以被序列化和反序列化，确保在网络上传输时能够正确识别和处理。



2.整体设计分析

```

J CancellableDelegatingFuture.java
J DistributedExecutorService.java
J ExecutionCallbackAdapter.java
J ExecutionCallbackAdapterFactory.java
J ExecutorDataSerializerHook.java
J ExecutorServiceProxy.java
J package-info.java
J RunnableAdapter.java
  
```

executor 整体对外接口如下: (LocalExecutorStats.java)

getCancelledTaskCount()	返回执行器服务上的已取消操作数。
getCompletedTaskCount()	返回执行器服务上已完成的操作数。
getPendingTaskCount()	返回执行器服务上的挂起操作数。
getStartedTaskCount()	返回执行器服务上的已启动操作数。
getTotalExecutionLatency()	返回已完成操作的总执行时间。
getTotalStartLatency()	返回已启动操作的总启动延迟。

(1) 整体设计分析-ExecutorDataSerializerHook

ExecutorDataSerializerHook 类的作用是提供和管理与任务执行相关的数据序列化。详细来说, ExecutorDataSerializerHook 类的主要作用是帮助 Hazelcast 系统识别和处理与任务执行相关的不同类型操作。它通过实现 DataSerializerHook 接口, 提供了一个方法来根据类型 ID 创建相应的操作对象 (比如任务操作、取消操作、关闭操作等)。这些操作对象通常是与分布式任务执行相关的, 而这个类通过序列化工厂将它们转化为可以在网络中传输的格式。

createFactory 方法: createFactory 方法返回一个 DataSerializableFactory 对象。DataSerializableFactory 是一个工厂接口, 它的作用是根据一个类型 ID 创建不同类型的对象。

通俗来说：

这个类就像一个“任务工厂”，内部调用了工厂方法，根据上述 5 个序列化变量，返回对应的方法。它根据类型 ID 创建不同的任务（如可调用任务、取消任务等），并且知道如何将它们转化为可以在 Hazelcast 系统中传输的格式。这样，在 Hazelcast 集群中，当不同的节点需要执行这些任务时，系统能够识别并正确处理这些任务类型。

```
public class ExecutorDataSerializerHook implements DataSerializerHook {

    public static final int F_ID = FactoryIdHelper.getFactoryId(EXECUTOR_DS_FACTORY, EXECUTOR_DS_FACTORY_ID);

    public static final int CALLABLE_TASK = 0;
    public static final int MEMBER_CALLABLE_TASK = 1;
    public static final int RUNNABLE_ADAPTER = 2;
    public static final int CANCELLATION = 3;
    public static final int SHUTDOWN = 4;

    @Override
    public int getFactoryId() {
        return F_ID;
    }

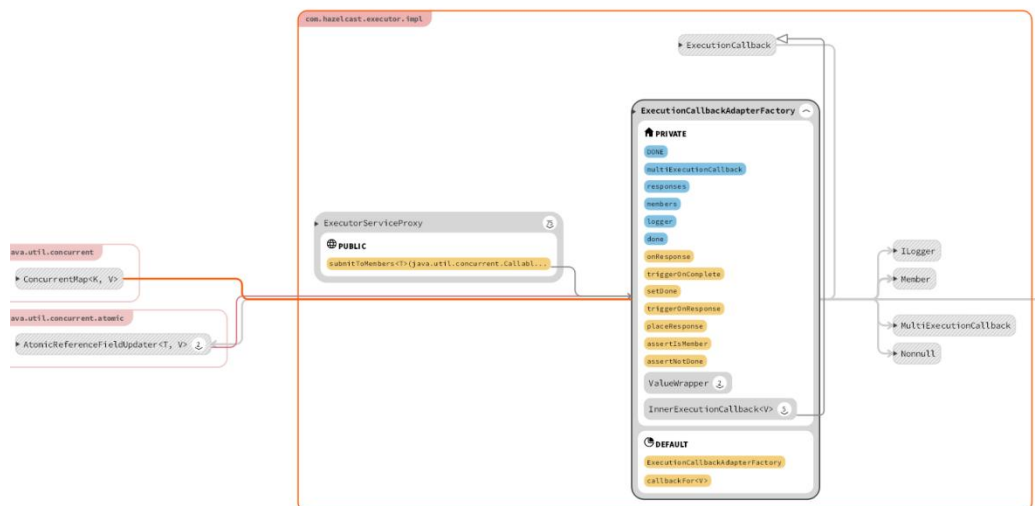
    @Override
    public DataSerializableFactory createFactory() {
        return typeId -> switch (typeId) {
            case CALLABLE_TASK -> new CallableTaskOperation();
            case MEMBER_CALLABLE_TASK -> new MemberCallableTaskOperation();
            case RUNNABLE_ADAPTER -> new RunnableAdapter<>();
            case CANCELLATION -> new CancellationOperation();
            case SHUTDOWN -> new ShutdownOperation();
            default -> null;
        };
    }
}
```

(2) 整体设计分析-ExecutionCallbackAdapterFactory

这是一个工厂类，用于创建任务执行的回调适配器。所谓回调（callback）就是在某个操作完成后自动执行的代码。这个工厂的目标是当任务在不同的机器（Member）上执行时，能够收集这些任务的执行结果，并在所有任务都完成后进行一些后续处理。

本质是对 ExecutionCallbackAdapterFactory.done 域进行了更新。系统为每个执行任务的成员创建一个回调对象，并在任务完成时：触发任务的回调，收集任务结果，当所有任务都完成后，触发最终的回调，通知任务已经全部完成。

采用了 AtomicReferenceFieldUpdater 方法，使得不用创建对象也可以对域进行更新。（注：AtomicReferenceFieldUpdater 是一个基于反射的工具类，它能对指定类的指定的 volatile 引用字段进行原子更新。）



onResponse 方法:

这个方法处理每个任务完成后的回调。它的工作是:

检查任务是否已完成, 如果完成了, 直接抛出异常。检查该成员是否有效, 确保只有参与执行的成员才能返回结果。触发 onResponse 回调, 把成员和它的执行结果传递给回调函数。保存任务结果, 并将它存储到 responses 集合中。检查所有任务是否完成, 如果所有任务都完成了, 就触发 onComplete。

triggerOnResponse 方法:

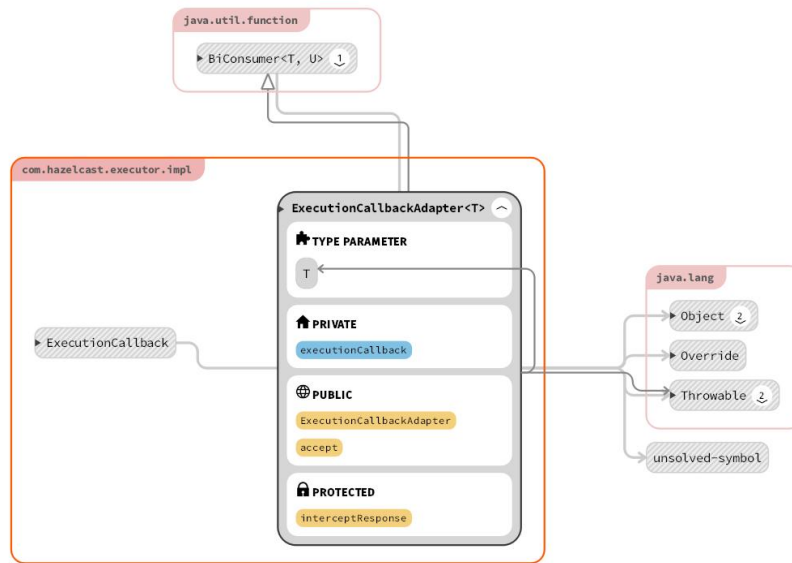
当某个成员的任务完成时, 这个方法会触发回调, 把任务结果 (response) 和任务执行的节点 (member) 传递给 multiExecutionCallback.onResponse, 如果发生异常, 记录警告信息。

triggerOnComplete 方法:

这个方法在所有任务完成后触发回调。当所有任务的结果都收集完后, multiExecutionCallback.onComplete(realResponses) 被调用, 传递一个包含所有成员及其任务结果的集合。

3) 整体设计分析-ExecutionCallbackAdapter

从 ExecutionCallbackAdapter 类名称就可以看出, 主要做了一个转换工作, 目的是将传统的回调接口 ExecutionCallback 转换成 Java 8 新提供的 CompletableFuture 使用的 BiConsumer, 这样就可以更方便地用更现代的方式处理异步任务。

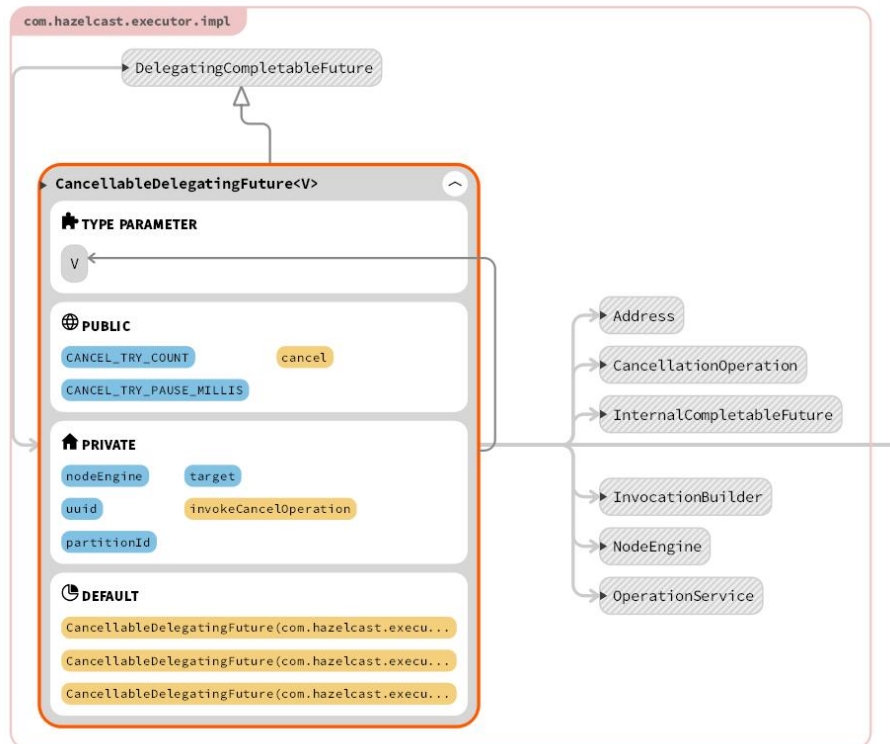


accept 方法:

这是 BiConsumer 接口的方法，用于接收异步任务的结果或异常。如果 Throwable（异常）为空，表示任务成功完成。调用 executionCallback 的 onResponse 方法，把结果传递过去。在这里可以先经过 interceptResponse 方法的处理。否则，表示任务出错误了。调用 executionCallback 的 onFailure 方法，把异常传递过去处理。

(4) 整体设计分析-CancellableDelegatingFuture

对于 cancellable 型任务，在这个类内部实现了 future 任务的删除 CancellableDelegatingFuture 是为了在 Hazelcast 分布式计算框架中支持取消异步操作而设计的。先来说说异步操作：在编程中，异步操作允许某些工作在后台运行，这样主程序就不需要等待它们完成，继续做其他事情，这种操作通常通过 Future 对象表示。CancellableDelegatingFuture，用于在后台任务运行时，允许你取消这个任务。它通过调度取消请求来中止任务，或在任务还在运行时尝试中断。



cancel 方法:

作用是在任务未完成前取消任务。首先检查任务是否完成，如果已完成，返回 false，表明取消无效。然后，通过 invokeCancelOperation 方法尝试取消该任务。然后等待并获取取消操作的结果（成功或失败）。如果取消成功或不成功，方法将返回相应的布尔值。如果过程中发生异常，会被捕获并处理。

核心功能实现方式为调用 invokeCancelOperation 方法:

创建一个 CancellationOperation 操作，用来提交取消请求。（uuid 用于标识请求，mayInterruptIfRunning 用于标识是否可以中断正在运行的任务）。使用 Hazelcast 的内部服务 OperationService 来创建并配置该取消操作。调用 invoke() 会发送这个请求来尝试取消任务，并返回一个 future 用于得到操作结果。

```

@Override
public boolean cancel(boolean mayInterruptIfRunning) {
    if (isDone()) {
        return false;
    }

    Future<Boolean> f = invokeCancelOperation(mayInterruptIfRunning);
    boolean cancelSuccessful = false;
    try {
        cancelSuccessful = f.get();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    } catch (Exception e) {
        throw rethrow(e);
    }

    completeExceptionally(new CancellationException());
    return cancelSuccessful;
}

```

更进一步，调用了内部方法中的取消部分，同时用了建造者模式来发送信息

```

private Future<Boolean> invokeCancelOperation(boolean mayInterruptIfRunning) {
    CancellationOperation op = new CancellationOperation(uuid, mayInterruptIfRunning);
    OperationService opService = nodeEngine.getOperationService();
    InvocationBuilder builder;
    if (partitionId > -1) {
        builder = opService.createInvocationBuilder(DistributedExecutorService.SERVICE_NAME, op, partitionId);
    } else {
        builder = opService.createInvocationBuilder(DistributedExecutorService.SERVICE_NAME, op, target);
    }
    builder.setTryCount(CANCEL_TRY_COUNT).setTryPauseMillis(CANCEL_TRY_PAUSE_MILLIS);
    return builder.invoke();
}

```

(5) 整体设计分析-RunnableAdapter

功能十分简单，将对象从无返回值的 runnable 变成有返回值的 callable。

(注：在 Java 中，有两种常见的接口用于表示可能会在后台执行的工作：Runnable 接口：这个接口很简单，只有一个 run() 方法，当你调用这个方法时，任务就会被执行。它不会返回结果，也不会抛出任何检查性异常。Callable 接口：和 Runnable 类似，但它的 call() 方法可以返回一个结果，并且可以抛出异常。通常用于需要返回值的任务。RunnableAdapter 这个类就是要把 Runnable 对象适配成一个 Callable 对象，以便一些需要返回值或异常处理的场合也可以用 Runnable。)

Q: RunnableAdapter 用在什么地方？

在分布式环境中，任务通常以 Callable 的形式提交，因为 Callable 能够返回结果并抛出异常，这是它比 Runnable 更多的功能。然而，不少任务最初是以 Runnable 形式存在。为了使这些任务能够被 Hazelcast 的执行服务处理，RunnableAdapter 提供了将其包装成 Callable 的功能。

在 Hazelcast 的 ExecutorService (如 ExecutorServiceProxy 类) 中，当用户提交一个 Runnable 时，可以通过 RunnableAdapter 将其内部转换为 Callable。这使得系统能够统一处理，无需区分处理 Runnable 和 Callable 的逻辑。

(6) 整体设计分析-DistributedExecutorService

DistributedExecutorService 类是 Hazelcast 框架中用于分布式任务执行的组件。这个组件的主要功能是管理和执行在分布式系统中的各种任务。这可以被比喻成一个“调度员”，负责将不同的任务分配给合适的工人（计算节点）去完成。

在 4.0 版本中 hazelcast 加入了 splitBrainProtectionAwareService 提供了数据的**脑裂保护**（在执行操作前检查集群最小可用的 member 数量，避免在网络分区条件下操作在集群的多数成员成功）

同样用原子方法实现了 CallableProcessor.responseFlag 的更新，并且实现了 executorConfigCache 功能 缓存了一部分 config 信息，加快查找速度。

核心程序为 execute 调用 processor 方法,具体分析见最后的高级设计意图分析部分。

(7) 整体设计分析-ExecutorServiceProxy

由于是代理部分，类及其复杂，uml 整理成表格形式如下：

修改器和类型	方法和描述
void	execute(Runnable command, MemberSelector memberSelector)在随机选择的成员上执行任务。
void	executeOnAllMembers(Runnable command)在所有已知群集成员上执行任务。
void	executeOnKeyOwner(Runnable command, Object key)在指定密钥的所有者上执行任务。
void	executeOnMember(Runnable command, Member member)在指定成员上执行任务。
void	executeOnMembers(Runnable command, Collection<Member> members)在每个指定成员上执行任务。
void	executeOnMembers(Runnable command, MemberSelector memberSelector)在每个选定成员上执行任务。
LocalExecutorStats	getLocalExecutorStats()返回与此执行器服务相关的本地统计信息。
<T> void	submit(Callable<T> task, ExecutionCallback<T> callback)将任务提交给随机成员。
<T> Future<T>	submit(Callable<T> task, MemberSelector memberSelector)将任务提交给随机选择的成员，并返回表示该任务的 Future。

修改器和类型	方法和描述
<T> void	submit(Callable<T> task, MemberSelector memberSelector, ExecutionCallback<T> callback)将任务提交给随机选择的成员。
<T> void	submit(Runnable task, ExecutionCallback<T> callback)将任务提交给随机成员。
<T> void	submit(Runnable task, MemberSelector memberSelector, ExecutionCallback<T> callback)将任务提交给随机选择的成员。
<T> Map<Member,Future<T>>	submitToAllMembers(Callable<T> task)将任务提交给所有群集成员，并返回成员-未来对的映射，表示每个成员上的任务即将完成。
<T> void	submitToAllMembers(Callable<T> task, MultiExecutionCallback callback)将任务提交给所有群集成员。
void	submitToAllMembers(Runnable task, MultiExecutionCallback callback)将任务提交给所有群集成员。
<T> Future<T>	submitToKeyOwner(Callable<T> task, Object key)将任务提交给指定密钥的所有者，并返回表示该任务的Future。
<T> void	submitToKeyOwner(Callable<T> task, Object key, ExecutionCallback<T> callback)将任务提交到指定密钥的所有者。
<T> void	submitToKeyOwner(Runnable task, Object key, ExecutionCallback<T> callback)将任务提交给指定密钥的所有者。
<T> Future<T>	submitToMember(Callable<T> task, Member member)将任务提交到指定成员并返回表示该任务的Future。
<T> void	submitToMember(Callable<T> task, Member member, ExecutionCallback<T> callback)将任务提交到指定成员。
<T> void	submitToMember(Runnable task, Member member, ExecutionCallback<T> callback)将任务提交到指定成员。
<T> Map<Member,Future<T>>	submitToMembers(Callable<T> task, Collection<Member> members)向给定成员提交任务，并返回成员-未来对的返回图，表示每个成员的任务即将完

修改器和类型	方法和描述
	成
<T> void	submitToMembers(Callable<T> task, Collection<Member> members, MultiExecutionCallback callback)将任务提交到指定成员。
<T> Map<Member,Future<T>>	submitToMembers(Callable<T> task, MemberSelector memberSelector)向选定的成员提交任务，并返回成员-未来对的映射，表示每个成员上的任务即将完成。
<T> void	submitToMembers(Callable<T> task, MemberSelector memberSelector, MultiExecutionCallback callback)将任务提交到所选成员。
void	submitToMembers(Runnable task, Collection<Member> members, MultiExecutionCallback callback)将任务提交到指定成员。
void	submitToMembers(Runnable task, MemberSelector memberSelector, MultiExecutionCallback callback)将任务提交到所选成员。

不同的 execute_on_xx 主要靠调用不同的 submit_to_xx 来实现

具体的实现细节会在高级设计意图分析中给出

Part 3.高级设计意图分析

1.ExecutorServiceProxy

executor 服务的分布式实现。

```
public class ExecutorServiceProxy
    extends AbstractDistributedObject<DistributedExecutorService>
    implements IExecutorService {
```

继承自抽象分布类（一个很抽象的类，只提供四个基本方法），向下实现了最终呈现给外部的接口：IExecutorService

其提供了其他方法，例如在特定成员上执行任务、在特定密钥的所有者上执行任务、在多个成员上执行任务以及使用回调侦听执行结果。

同时支持裂脑保护分离脑保护 其中的配置文件在集群版本 3.10 及更高版本

次要方法：

check sync 用于阻止不在进行中的任务重载当前系统，通过计算时间的方式，如果不在当前允许时间范围，set 为 0，阻止任务提交

```
private boolean checkSync() {
    boolean sync = false;
    long last = lastSubmitTime;
    long now = Clock.currentTimeMillis();
    if (last + SYNC_DELAY_MS < now) {
        CONSECUTIVE_SUBMITS.set(this, 0);
    } else if (CONSECUTIVE_SUBMITS.incrementAndGet(this) % SYNC_FREQUENCY == 0) {
        sync = true;
    }
    lastSubmitTime = now;
    return sync;
}
```

根据键值获取分进程 id 号：

```
private <T> int getTaskPartitionId(Callable<T> task) {
    if (task instanceof PartitionAware) {
        Object partitionKey = ((PartitionAware) task).getPartitionKey();
        if (partitionKey != null) {
            return getNodeEngine().getPartitionService().getPartitionId(partitionKey);
        }
    }
    return random.nextInt(partitionCount);
}
```

invoke 方法：将任务加入对应的队列，包括带时间参数和不带时间参数的两个版本，其中带时间参数的还涉及到了 wait 函数。

带时间参数的 invoke 方法中，较为关键的 wait 函数：

```
private <T> boolean wait(long timeoutNanos, List<Future<T>> futures) throws InterruptedException {
    boolean done = true;
    for (int i = 0, size = futures.size(); i < size; i++) {
        long startNanos = Timer.nanos();
        Object value;
        try {
            Future<T> future = futures.get(i); // get 方法拥有"状态依赖"的内在特性，因而调用者不需要知道任务的状态，此外在任务提交和获得结果中包含的安全发布属性也确保了这个方法是线程安全的。Future.get 的异常处理代码将处理两个可能的问题：任务遇到一个 Exception，时间片耗尽。
            value = future.get(timeoutNanos, TimeUnit.NANOSECONDS);
        } catch (ExecutionException e) {
```

```

        value = e;
    } catch (TimeoutException e) { //时间片耗尽，则返回失败

        done = false;

        for (int l = i; l < size; l++) {
            Future<T> f = futures.get(i);

            if (f.isDone()) { //若部分完成，则将已完成的任务放入 future 中
                futures.set(l, completedSynchronously(f,
getNodeEngine().getSerializationService()));
            }
        }

        break;
    }

    futures.set(i, InternalCompletableFuture.newCompletedFuture(value));
    timeoutNanos -= Timer.nanosElapsed(startNanos);
}

return done;
}

```

cancel ALL 调用了 future 中的 cancel 方法取消所有 list 中的 future 型任务，较为简单，就不再多提。invoke All 由于任意执行，所以只需要 throw 出奇怪的例外处理就行，内部就一行后续还有很多细碎的判断方法，这里略去，基本看一眼就知道他在分析什么。

值得注意的方法： shutdown 和 shutdownNow:

shutdown: 停止，先将任务提交到关闭操作（后续调用方法包中的 shutdown 方法关闭），最后用最开始的关闭例外处理传给 waitAll 处理，相当于正在执行的任务会继续执行下去，没有被执行的则中断。

shutdownNow: 最后额外加了返回一个空表，相当于强行停止所有任务，正在执行的任务也会被直接中断。

```

public void shutdown() {
    NodeEngine nodeEngine = getNodeEngine();
    Collection<Member> members = nodeEngine.getClusterService().getMembers();
    OperationService operationService = nodeEngine.getOperationService();
    Collection<Future> calls = new LinkedList<>();

    for (Member member : members) {
        Future f = submitShutdownOperation(operationService, member);
        calls.add(f);
    }

    waitWithDeadline(calls, 3, TimeUnit.SECONDS, shutdownExceptionHandler); //
    //Calculate timeouts for whole operation and per future. If corresponding TimeUnits not set assume
    // the default of TimeUnit.SECONDS
}

```

```

        private InvocationFuture<Object> submitShutdownOperation(OperationService operationService, Member
member) {
            ShutdownOperation op = new ShutdownOperation(name);
            return operationService.invokeOnTarget(getServiceName(), op, member.getAddress());
        }/*****
****/
        @Nonnull
        @Override
        public List<Runnable> shutdownNow() {
            shutdown();
            return Collections.emptyList();
        }
    }

```

最后是一个选择成员的函数，其中 select 方法经过笔者四层的深度搜索，发现在 cluster.MemberSelector.java 方法中给出了一定的注释的 select 但是似乎也没什么特殊的选择，可能就是一种随机的选择，重点在于选择的时候进行了附加的判断，而不是选择的方式：

```

        private List<Member> selectMembers(@Nonnull MemberSelector memberSelector) {
            checkNotNull(memberSelector, "memberSelector must not be null");
            List<Member> selected = new ArrayList<>();
            Collection<Member> members = getNodeEngine().getClusterService().getMembers();
            for (Member member : members) {
                if (memberSelector.select(member)) { //以为很特殊，但实际很正常
                    selected.add(member);
                }
            }
            if (selected.isEmpty()) {
                throw new RejectedExecutionException("No member selected with memberSelector[" +
memberSelector + "]); //错误处理
            }
            return selected;
        }

        public interface MemberSelector {

            /**
             * Decides if the given member will be part of an operation or not.
             *
             * @param member
             * the member instance to decide upon
             * @return true if the member should take part in the operation,
             * false otherwise
             */
            boolean select(Member member);
        }
    }

```

主要方法：

根据 execute 中不同参数决定在怎样的成员上执行任务，根据 submit 中不同参数决定将任务提交给哪些成员。但 execute 返回值倒是 void 所以只有 6 种，但 submit 返回值也有可能是 future 型 或者 map 型 所以总共有多达 20 种不同的 submit 方法。

代码高度相似，但主要通过**编写主方法，剩余方法改写的方式实现**，以 submit to membe 为例：

首先编写了主方法，其中包含了各种可能用上的信息，包括 uuid 实际地址，节点启动，等同时根据 sync 检测信息给出了报错信息参考上面的方法列表，三种不同的方法都通过 @override 方式实现覆盖，减少了代码量，同时减少后期维护成本。

```
private <T> void submitToMember(@Nonnull Data taskData,
                                @Nonnull Member member,
                                @Nullable ExecutionCallback<T> callback) {

    checkNotNull(member, "member must not be null");
    checkNotShutdown();

    NodeEngine nodeEngine = getNodeEngine();
    UUID uuid = newUnsecureUUID();
    MemberCallableTaskOperation op = new MemberCallableTaskOperation(name, uuid, taskData);
    OperationService operationService = nodeEngine.getOperationService();
    Address address = member.getAddress();
    //完成了对象的创建
    InvocationFuture<T> future = operationService
        .createInvocationBuilder(DistributedExecutorService.SERVICE_NAME, op, address)
        .invoke();
    if (callback != null) {
        future.whenCompleteAsync(new ExecutionCallbackAdapter<>(callback))
            .whenCompleteAsync((v, t) -> {
                if (t instanceof RejectedExecutionException) {
                    callback.onFailure(t);
                } //错误处理
            });
    }
}

@Override
public <T> void submitToMember(@Nonnull Callable<T> task,
                                @Nonnull Member member,
                                @Nullable ExecutionCallback<T> callback) {

    checkNotNull(task, "task must not be null");
    checkNotShutdown();

    Data taskData = getNodeEngine().toData(task);
    submitToMember(taskData, member, callback);
}
```

2.DistributedExecutorService

为分布式实现的基础，在 proxy 中调用最后呈现给使用者,分析主要结合代码为主

前置知识: ConcurrentHashMap

ConcurrentHashMap 是 Java1.5 中引用的一个线程安全的支持高并发的 HashMap 集合类。

其方法和特点为：采用了非常精妙的"分段锁"策略，ConcurrentHashMap 的主干是个 Segment 数组。Segment 继承了 ReentrantLock，所以它就是一种可重入锁 (ReentrantLock)。在 ConcurrentHashMap，一个 Segment 就是一个子哈希表，Segment 里维护了一个 HashEntry 数组，并发环境下，对于不同 Segment 的数据进行操作是不用考虑锁竞争的。

比较常用的方法：

ConcurrentHashMap 的 put 方法：根据 hash 值找到对应的 segment，segment 是分段锁。

ConcurrentHashMap 的 get 方法: count>0,hash 找到 HashEntry,hash 相等并且 key 相同，若取 value 为 null，加锁重新获取。

ConcurrentHashMap 的 remove 方法：加锁，每删除一个元素就将那之前的元素克隆一边。因为设置为第一次 next 之后不能再改变。

ConcurrentHashMap 的 size () 方法: 2 次不锁住 segment 方式统计各个 segment 的大小，若 count 发生变化，采用加锁方式统计。modCount 变量，在 put，remove 和 clean 方法里操作元素，modcount 加 1。

前置知识：裂脑保护

所谓的脑裂，就是指在主从集群中，同时有两个主节点，它们都能接收写请求。而脑裂最直接的影响，就是客户端不知道应该往哪个主节点写入数据，结果就是不同的客户端会往不同的主节点上写入数据。而且，严重的话，脑裂会进一步导致数据丢失。

hazelcast 提供了一种保护方法，可以防止这种裂脑的发生。

CallableProcessor.responseFlag 的更新：

```
private static final AtomicReferenceFieldUpdater<Processor, Boolean> RESPONSE_FLAG =
    AtomicReferenceFieldUpdater.newUpdater(Processor.class, Boolean.class, "responseFlag");

// package-local access to allow test to inspect the map's values
final ConcurrentMap<String, ExecutorConfig> executorConfigCache = new ConcurrentHashMap<String,
ExecutorConfig>();
```

定义对象：

定义的所有对象如下：其中 shutdownExecuto 使用的方式较为巧妙 是 jdk1.6 中加入的新特性


```

private NodeEngine nodeEngine;
private ExecutionService executionService;

private final ConcurrentMap<UUID, Processor> submittedTasks = new ConcurrentHashMap<>();//利用前述
的 concurrenthashmap 创建提交任务

private final Set<String> shutdownExecutors
    = Collections.newSetFromMap(new ConcurrentHashMap<>());

//关闭方法 生成对 Map 进行包装的 Set。

//这个 Set 和被包装的 Map 拥有相同的 key 顺序（遍历 Set 调用的还是 Map 的 keySet），相同的并发特性（也就是说如
果对 ConcurrentHashMap 进行包装，得到的 Set 也将线程安全）。

//本质上来说，这个工厂方法（newSetFromMap）就是提供了一个和 Map 实现相对应的 Set 实现。

private final ExecutorStats executorStats = new ExecutorStats();//创建 executor 开始的总类
private final ConcurrentMap<String, Object> splitBrainProtectionConfigCache = new
ConcurrentHashMap<String, Object>();

private final ContextMutexFactory splitBrainProtectionConfigCacheMutexFactory = new
ContextMutexFactory();

private final ConstructorFunction<String, Object> splitBrainProtectionConfigConstructor =
    new ConstructorFunction<String, Object>() {
        @Override
        public Object createNew(String name) {
            ExecutorConfig executorConfig = nodeEngine.getConfig().findExecutorConfig(name);
            String splitBrainProtectionName = executorConfig.getSplitBrainProtectionName();
            return splitBrainProtectionName == null ? NULL_OBJECT : splitBrainProtectionName;
        }
    };

//上面三个对象都是脑裂保护用
private ILogger logger;

```

次要方法：

最开始的初始化任务，初始化了节点、整体服务程序、日志以及调用 metric 库的监控系统的初始化

```

@Override
public void init(NodeEngine nodeEngine, Properties properties) {
    this.nodeEngine = nodeEngine;
    this.executionService = nodeEngine.getExecutionService();
    this.logger = nodeEngine.getLogger(DistributedExecutorService.class);

    ((NodeEngineImpl) nodeEngine).getMetricsRegistry().registerDynamicMetricsProvider(this);
}

```

封装了 shutdown 时调用的 reset 函数，主要功能是清空所有的线程池内容(包括 cache)，重置 executor 启动部分等，代码如下

```

public void reset() {

```

```

        shutdownExecutors.clear();
        submittedTasks.clear();
        executorStats.clear();
        executorConfigCache.clear();
    }

```

分布式执行时，cancel 方法通过前置知识中的 concurrentmap 中的 remove 在集合中取消掉这一个进程的位置；再进行多次判断，需要在确实进程不空+收到了取消例外的标志+数据有效的情况下才真正取消

```

public boolean cancel(UUID uuid, boolean interrupt) {
    Processor processor = submittedTasks.remove(uuid); //调用 remove
    if (processor != null && processor.cancel(interrupt)) {
        if (processor.sendResponse(new CancellationException())) {
            if (processor.isStatisticsEnabled()) {
                executorStats.cancelExecution(processor.name);
            }
            return true; //仅当
        }
    }
    return false;
}

```

shutdownExecutor 中调用 java 库函数的取消，同时在进程 config_cache 中移除该进程：

```

public void shutdownExecutor(String name) {
    executionService.shutdownExecutor(name);
    shutdownExecutors.add(name);
    executorConfigCache.remove(name);
}

```

同时与代理的交互主要通过创建分布式对象完成：

```

@Override
public ExecutorServiceProxy createDistributedObject(String name, UUID source, boolean local) {
    return new ExecutorServiceProxy(name, nodeEngine, this);
}

```

与前述的初始化任务相对对应的是摧毁进程，主要操作基本类似，去除所占空间，最后取消裂脑保护：

```

@Override
public void destroyDistributedObject(String name, boolean local) {
    shutdownExecutors.remove(name);
    executionService.shutdownExecutor(name);
    executorStats.removeStats(name);
    executorConfigCache.remove(name);
}

```

```
splitBrainProtectionConfigCache.remove(name);  
}
```

由于引入了 cache 系统，当需要查找 config 信息的时候要么选择查找，要么选择创建并将其加入缓存池，代码及注释如下：

```
/**  
 * Locate the {@code ExecutorConfig} in local {@link  
 * #executorConfigCache} or find it from {@link  
 * NodeEngine#getConfig()} and cache it locally.  
 */  
private ExecutorConfig getOrFindExecutorConfig(String name) {  
    ExecutorConfig cfg = executorConfigCache.get(name);  
    if (cfg != null) {  
        return cfg;  
    } else {  
        cfg = nodeEngine.getConfig().findExecutorConfig(name);  
        ExecutorConfig executorConfig = executorConfigCache.putIfAbsent(name, cfg);  
        return executorConfig == null ? cfg : executorConfig;  
    }  
}
```

有关裂脑保护的主要方法：

Processor 方法：继承自 java 库函数

```
private final class Processor extends FutureTask implements Runnable {
```

内部参数如下：

```
//is being used through the RESPONSE_FLAG. Can't be private due to reflection constraint.  
volatile Boolean responseFlag = Boolean.FALSE;  
  
private final String name;  
private final UUID uuid;  
private final Operation op;  
private final String taskToString;  
private final long creationTime = Clock.currentTimeMillis();  
private final boolean statisticsEnabled;
```

提供了**两种构造方法**，分别对应 runnable 和 callable 类型（代码仅展示一种）

```
private Processor(String name, UUID uuid,  
                  @Nonnull Callable callable,  
                  Operation op,  
                  boolean statisticsEnabled) {
```

```

        //noinspection unchecked
        super(callable);

        this.name = name;

        this.uuid = uuid;

        this.taskToString = String.valueOf(callable);

        this.op = op;

        this.statisticsEnabled = statisticsEnabled;
    }

```

核心函数 run 函数中调用监察函数计时，返回给最外部的接口 executorStats 同时也处理了例外，具体处理方式见下述代码及注释：

```

@Override
public void run() {
    long start = Clock.currentTimeMillis();//计时开始
    if (statisticsEnabled) {
        executorStats.startExecution(name, start - creationTime);
    }//数据可用，记录开始执行
    Object result = null;
    try {
        super.run();

        if (!isCancelled()) {
            result = get();
        }//非取消情况，得到结果
    } catch (Exception e) {
        logException(e);
        result = e;
    } //否则，得到例外信息，保存至结果
    finally { //说明运行结束 or 取消，反正进程结束了
        if (uuid != null) {
            submittedTasks.remove(uuid);
        }

        if (!isCancelled()) {
            sendResponse(result);

            if (statisticsEnabled) {
                executorStats.finishExecution(name, Clock.currentTimeMillis() - start);
            }//非取消，向外传递结果，同时记录结束执行到最外层接口
        }
    }
}

```

、还有三个小的分方法，涉及到错误日志和结果的发送，内容及实现方式直接在代码框中给出：

```

private void logException(Exception e) {

```

```

        if (logger.isFinestEnabled()) {
            logger.finest("While executing callable: " + taskToString, e);
        }
    } //发生例外时的日志,

    private boolean sendResponse(Object result) {
        if (RESPONSE_FLAG.compareAndSet(this, Boolean.FALSE, Boolean.TRUE)) {
            try {
                op.sendResponse(result);
            } catch (HazelcastSerializationException e) {
                op.sendResponse(e);
            }
            return true;
        }
        return false;
    } //发出结果, 根据是否发生例外来选择发送结果/例外信息

    boolean isStatisticsEnabled() {
        return statisticsEnabled;
    } //返回布尔值, 在外部函数调用时在, 这个值由任务 cfg 信息确定

```

execute 方法：调用 processor 方法，完成最终运行

由于缓存 cache 的加入 需要用 getOrFindExecutorConfig(name) 获取 cfg 信息根据任务类型，二选一完成方法调用尝试执行，如果返回错误，则向外发出错误信息类型，并将该 uuid 号从哈希表中移除（加锁）

```

public <T> void execute(String name, UUID uuid,
                        @NonNull T task, Operation op) {
    ExecutorConfig cfg = getOrFindExecutorConfig(name);
    if (cfg.isStatisticsEnabled()) {
        executorStats.startPending(name);
    }
    Processor processor;
    if (task instanceof Runnable) {
        processor = new Processor(name, uuid, (Runnable) task, op, cfg.isStatisticsEnabled());
    } else {
        processor = new Processor(name, uuid, (Callable) task, op, cfg.isStatisticsEnabled());
    } //注意数据是否有效是保存在 cfg 中

    if (uuid != null) {
        submittedTasks.put(uuid, processor);
    } //二选一

    try {
        executionService.execute(name, processor);
    }
}

```

```
    } catch (RejectedExecutionException e) {  
        if (cfg.isStatisticsEnabled()) {  
            executorStats.rejectExecution(name);  
        }  
        logger.warning("While executing " + task + " on Executor[" + name + "]", e);  
        if (uuid != null) {  
            submittedTasks.remove(uuid);  
        }  
        processor.sendResponse(e);  
    }  
}
```

Part 4.设计模式分析

1.设计模式分析-工厂模式

hazelcast.ExecutorDataSerializerHook 中实现了工厂化方法。

根据工厂化的定义，由子类决定实例化哪一个类，这里可以看到是根据 typeId(0~4)来决定具体实例化哪一个方法；同时也明显的出现了 factory 标志

```

public class ExecutorDataSerializerHook implements DataSerializerHook {

    public static final int F_ID = FactoryIdHelper.getFactoryId(EXECUTOR_DS_FACTORY, EXECUTOR_DS_FACTORY_ID);

    public static final int CALLABLE_TASK = 0;
    public static final int MEMBER_CALLABLE_TASK = 1;
    public static final int RUNNABLE_ADAPTER = 2;
    public static final int CANCELLATION = 3;
    public static final int SHUTDOWN = 4;

    @Override
    public int getFactoryId() {
        return F_ID;
    }

    @Override
    public DataSerializableFactory createFactory() {
        return typeId -> switch (typeId) {
            case CALLABLE_TASK -> new CallableTaskOperation();
            case MEMBER_CALLABLE_TASK -> new MemberCallableTaskOperation();
            case RUNNABLE_ADAPTER -> new RunnableAdapter<>();
            case CANCELLATION -> new CancellationOperation();
            case SHUTDOWN -> new ShutdownOperation();
            default -> null;
        };
    }
}

```

ExecutionCallbackAdapterFactory 中，最后的对内部的接口中出现：
如果成功返回的是正常的答复，否则返回的是错误信息。

```

private final class InnerExecutionCallback<V> implements ExecutionCallback<V> {
    private final Member member;

    private InnerExecutionCallback(Member member) {
        this.member = member;
    }

    @Override
    public void onResponse(V response) {
        ExecutionCallbackAdapterFactory.this.onResponse(member, response);
    }

    @Override
    public void onFailure(Throwable t) {
        ExecutionCallbackAdapterFactory.this.onResponse(member, t);
    }
}

```

2.设计模式分析-Adapter 适配器模式

ExecutionCallbackAdapter:

将传统的回调接口 ExecutionCallback 转换成 Java 8 新提供的 CompletableFuture 使用的 BiConsumer，这样就可以更方便地用更现代的方式处理异步任务,使得其可以和

`{@link java.util.concurrent.CompletableFuture#whenComplete(BiConsumer)}`

结合而被直接调用


```

public class ExecutionCallbackAdapter<T> implements BiConsumer<T, Throwable> {

    private final ExecutionCallback<T> executionCallback;

    public ExecutionCallbackAdapter(ExecutionCallback<T> executionCallback) {
        this.executionCallback = executionCallback;
    }

    @Override
    public final void accept(T response, Throwable t) {
        if (t == null) {
            executionCallback.onResponse((T) interceptResponse(response));
        } else {
            executionCallback.onFailure(t);
        }
    }

    protected Object interceptResponse(Object o) {
        return o;
    }
}

```

RunnableAdapter:

将对象从无返回值的 runnable 变成有返回值的 callable。

3.设计模式分析-Proxy 代理模式

在整个 ExecutorServiceProxy 中，通过构建大量的 execute submit(返回值不同/提交 or 执行的对象不同等等)，实现了在分布式系统上的并发计算，根据 execute 中不同参数决定在怎样的成员上执行任务，根据 submit 中不同参数决定将任务提交给哪些成员。主要实现方式在高级设计意图的部分中有主要提及。